
GPT-2 BENCHMARK TECHNICAL REPORT

David Korcak

ETH Zürich

Switzerland

x.com/thatdave_k

github.com/edavidk7/NoCap-Test

dkorcak@ethz.ch

ABSTRACT

In this report, I outline the different approaches taken while trying to speed up the GPT-2 124M model training to reach the target validation loss on the FineWeb dataset *without changing any of the hyper-parameters*. My first ideas were focused purely on the architecture of the model, trying to improve expressivity, reduce FLOPs, or both. I then resorted to changing the attention mask, better exploiting the training-data distribution. By observing the convergence behavior at different gradient noise levels (physical batch sizes), I was able to tune the training process to maximize token (and time) efficiency, with an additional passive EMA on model weights. Finally, an auxiliary loss term was introduced, further improving convergence speed. The final achieved speedup was ~25%.

1 AI USAGE DECLARATION

To perform my experiments, I heavily relied on **Claude Code** serving as an intelligent launch script & training watchdog. Additionally, I used it to implement some of the training modifications (EMA) when I was travelling and only had my phone available. It was necessary for me to finish the challenge in time and without excessively high costs, as I ran all of the experiments on an Amazon EC2 instance (RTX6000 Pro Blackwell) which costs about \$4.5/hour, and **Claude** could launch and debug experiments per my instructions overnight. Sometimes, I would consult Claude on my ideas, especially when I wanted to know whether related literature already exists.

2 A COMMENT ON STOCHASTICITY

Before running any performance experiments, I implemented the functionality to manually set the random seed of all libraries. This had to be done, as even some of the runs of the **baseline** as shipped in the original repository **would not cross the target loss of 3.3821**. I then reran the baseline with seed 42, which produced a valid run achieving the target loss, and was used **for all** downstream experiments. It may be a part of future work to try and reproduce the results with a random seed sweep, however, I had only a limited budget for GPU hours and chose to rather invest them in actual performance tuning and experimentation rather than random seed sweeps.

3 ARCHITECTURAL CHANGES

My proposed architectural changes were motivated by the fact that we train & evaluate on FineWeb, a very diverse dataset, where the model has to tackle/predict various topics. I was either trying to improve the expressivity without adding too many parameters/FLOPs, or cutting FLOPs without reducing expressivity too much. Some of them failed completely, and some have shown to improve the final validation loss.

3.1 LEARNABLE PER-HEAD PER-BLOCK SOFTMAX TEMPERATURE

Initially, I was trying to “emulate” a Mixture-of-Experts architecture, by allowing further, learnable specialization of each head in each layer. One way to achieve that was (in my eyes) a learnable temperature parameter. The intention is to allow each head to learn the entropy of its attention softmax distribution to best reason over the data.

Intuitively, I expected to see the learning procedure adjust the temperatures, with lower magnitudes in the early layers (vast knowledge summary) and higher magnitudes near the final layer, where the model summarizes only few, high-confidence embeddings to produce the final token prediction.

To make the implementation numerically feasible, constrainable, and to create a proper structural prior for the parameter, the gradient was computed towards a log-temperature value, passed through a `tanh` squish and scaled to a limited range. Formally:

$$\text{softmax}(\mathbf{x}_i, \tau) = \frac{e^{\tau \mathbf{x}_i}}{\sum_{j=1}^n e^{\tau \mathbf{x}_j}}, \mathbf{x} \in \mathbb{R}^n \text{ and } \tau = e^{c \tanh(p)} \quad (1)$$

where p is a learnable parameter, $c \in [0.2, 0.6]$

Unfortunately, this failed to produce any meaningful improvement, the model’s validation loss evolution during training was identical to the baseline, while the computation time went up due to a slight increase in FLOPs. An interesting observation was that initially, the temperatures followed the trend I had predicted (first block low, last block high), however, over time all magnitudes slowly crashed to the floor defined by the `tanh` clamp. My conjecture is that gradient descent is powerful enough to modulate the activation magnitude by scaling up the relevant **Query** matrix’s norm, and does not need the additional parameter.

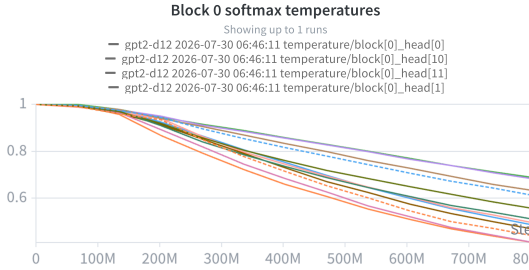


Figure 1: Learned softmax temperatures for individual heads in transformer **block 0** over the course of training.

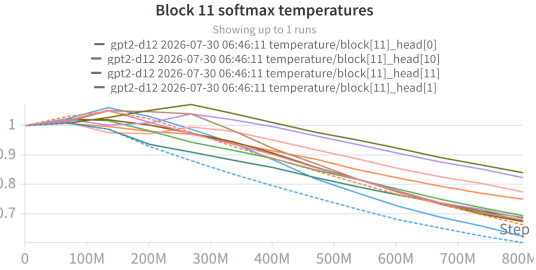


Figure 2: Learned softmax temperatures for individual heads in transformer **block 11** over the course of training.

Because this change did not bring any noticeable improvement in loss, slightly increased the overall wall clock time, and can be side-stepped by the optimization procedure, I decided to abandon it.

3.2 LEAVE-EVERY-OTHER-OUT MLP REMOVAL

Since the dataset requires memorization of significant amount of information, it was intriguing to reason about the *minimum* amount of “memory capacity” a model of this size/dimensionality needs to store that knowledge, and if perhaps the baseline architecture contains unnecessary parameters. Based on the findings in “Transformer Feed-Forward Layers Are Key-Value Memories (Geva et al., 2021)”, I tried out simply removing every other MLP from the baseline model, but keeping the attention layer and empty **RMSNorm** path in each block (empirically found to improve convergence).

Variant	FFN	Act.	α	Hidden	FFN blocks	Val loss	Wall	Proj. target
baseline	MLP	gelu	4	3072	12 of 12	3.379	149.6 min	-
SwiGLU	GLU	silu	4	2048	12 of 12	3.409	133.7 min	158.9 min
GEGLU	GLU	gelu	4	2048	12 of 12	3.408	143.9 min	170.1 min
drop MLP	MLP	gelu	4	3072	6 of 12	3.458	104.8 min	170.0 min
drop GLU	GLU	silu	4	2048	6 of 12	3.472	106.1 min	188.5 min
drop GLU wider	GLU	silu	6	3072	6 of 12	3.442	118.2 min	173.8 min

Table 1: Comparison of different MLP configurations, hyperparameters **identical** to baseline. Projected target refers to the time estimated to reach the accepted val. loss threshold.

Without any further changes, this led up to about 21% forward FLOP reduction. However, expressivity of the model was clearly impacted. I came up with two potential resolutions. First, I tried swapping standard MLPs for Gated Linear Unit architectures (GLU, Shazeer (2020)), which, in the past, I experimentally found to be more stable and faster learners, as well as more efficient information stores. Then, to buy back the parameters, but in a way that is easier to parallelize on the GPU with less overhead, I varied the MLP width scaling factor, denoted α . Instead of more small MLPs, I kept fewer but wider.

Variant	Fwd GFLOP/token	vs baseline	FFN GFLOP/token
baseline	0.266	+0.0%	0.113
SwiGLU	0.266	+0.0%	0.113
GEGLU	0.266	+0.0%	0.113
drop MLP	0.209	-21.3%	0.057
drop GLU	0.209	-21.3%	0.057
drop GLU wider	0.238	-10.6%	0.085

Table 2: Comparison of compute requirements of every FFN configuration tested.

Furthermore, I tested one configuration where the α parameter varied 4-3 following the leave-one-out fashion, however, this configuration didn’t help either and led to a projected slower run. Because all attempts failed, I also abandoned this route.

3.3 LM HEAD BOTTLENECK (EMBEDDING FACTORIZATION)

Because the model’s input embedding matrix $\mathbf{E}$ is typically of dimension $V \times n_{\text{embd}}$, for the final cross-entropy/perplexity calculation, there is exactly $V \times T$ dot products of dim n_{embd} . My idea for obtaining a lower runtime was to reduce n_{embd} , but without significantly affecting the model’s internal capacity. To this extent, I added additional input/output projection matrices $\mathbf{W}_{\text{bottleneck}}^i$ and $\mathbf{W}_{\text{bottleneck}}^o$ of dimensions $n_{\text{bottleneck}} \times n_{\text{embd}}$ and vice versa. Then $\mathbf{E} \in \mathbb{R}^{V \times n_{\text{bottleneck}}}$, and the output LM head is just transpose of $\mathbf{E}$. This modification saves a noticeable number of FLOPs, because the full-vocabulary dot product is computed over a (significantly) reduced dimension. Suppose $V = 50257$ and $n_{\text{embd}} = 768$ (GPT-2 124M). If $n_{\text{bottleneck}} = 512$, then the FLOPs (percentually) decrease to $(V \times 2 \cdot 512 \times T + 2 \cdot 512 \cdot 768 \times T) / (V \times 2 \cdot 768 \times T)$. Simplified, $(V \times 2 \cdot 512 + 2 \cdot 512 \cdot 768) / (V \times 2 \cdot 768) \approx 0.68 = 68\%$. A 32% reduction in FLOPs is quite significant, and because the LM head constitutes a significant number of the model’s overall forward FLOPs, the total number of FLOPs decreases by about 9%.

Variant	$n_{\text{bottleneck}}$	Iters	Val loss	Wall	Proj. target
baseline	–	10566	3.379	117.0 min	–
bottleneck 576	576	10566	3.399	112.3 min	126.9 min
bottleneck 512	512	10566	3.408	103.8 min	124.3 min

Table 3: Comparison of $V \times n_{\text{embd}}$ **tied-embedding** head configurations against a matching-schedule baseline. The baseline in this case **was not** the vanilla baseline, but a run tuned mid-way through my experimentation

While the introduction of a bottleneck dimension helped reduce the absolute wall clock time noticeably, the model always lost expressivity, to an unrecoverable extent. All projections would lead to a longer runtime than the then-current baseline. Later, SVD decomposition of $\mathbf{E}$ from the baseline revealed ~ 740 non-zero singular values, indicating the baseline is already on the verge of using the full embedding rank. That analysis essentially gave a lower bound, and prompted me to stop this research direction.

3.4 PER-HEAD, DATA-DRIVEN LIMITED CONTEXT LENGTH

Lastly, since the README mentioned successful attempts that change the model’s context length, I got an idea to hard-mask the context in each individual head of the attention block. The idea was derived from the *learnable temperature* experiment, except the per-head specialization is manually selected and fixed to attend only some K tokens in the past. The head would then learn to accurately summarize only over a short context, without having to model all of the dependencies.

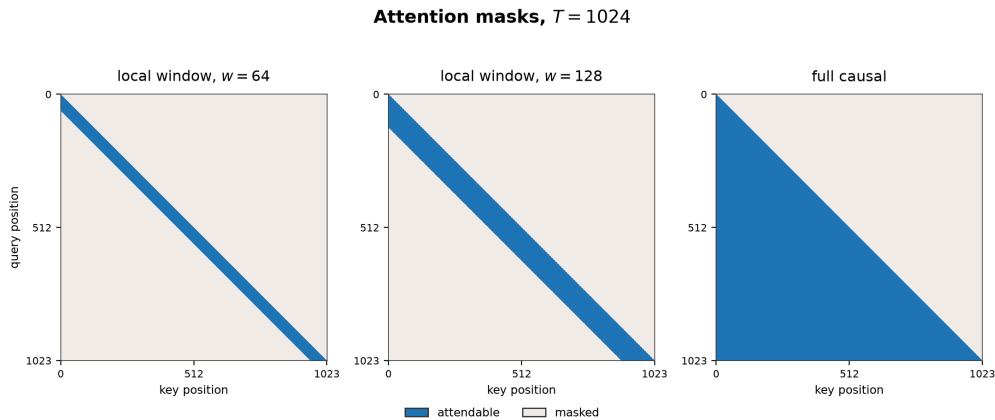


Figure 3: Attention masks for different context window sizes.

I implemented this using PyTorch’s `flex_attention`, which allows block-wise computation, and therefore can exploit the reduction in necessary FLOPs that comes from a limited context window. The first window sizes were picked manually to be 64 and 128, somewhat intuitively based on the expected English language sentence length and tokenization ratio (word $\rightarrow$ #tokens). This change seemed to have helped significantly, reducing both the training time and improving convergence to lower loss values. I wondered as to why these parameters worked so well (and if there are even better ones) and got an idea to measure the **sentence lengths** and look at their distribution on both the training and validation dataset. After some research, **token copy distance** (distance to the nearest prior occurrence of the same token) seemed like a more suitable metric, roughly signaling how far into the past the model needs to attend for NTP.

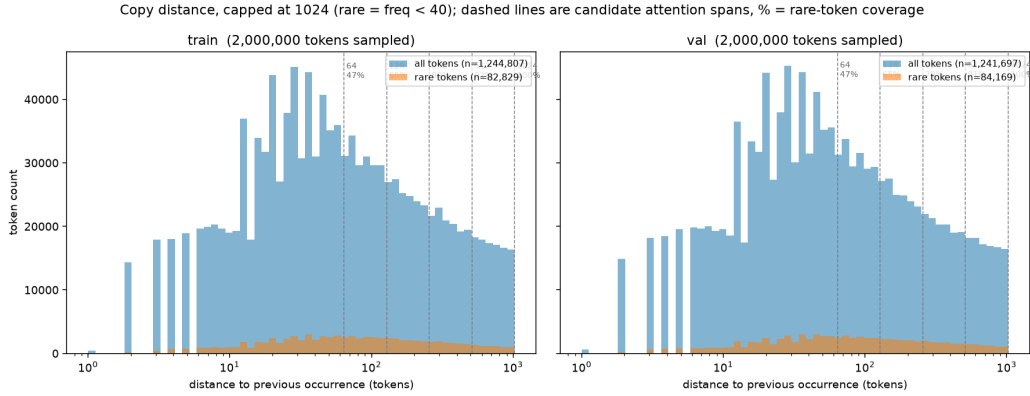


Figure 4: Distribution of token copy distances on the training and validation datasets.

The distributions show that over 40% of token copy distances are below 64, so a $w = 64$ head already covers the majority of the dependencies it is likely to be asked about, and the remaining spans are chosen so that the heads left at wider windows cover the tail. Expressivity on this specific dataset is therefore preserved, while the short windows likely add a useful inductive bias, and slightly reduce the forward pass time.

Attention span (head:span, unnamed = full ctx.)	Iters	Val loss
full causal (no windowing)	11575	3.377
0:64,1:64,2:64,3:64,4:64,5:64	11575	3.365
0:64,1:64,2:64,3:64,4:128,5:128	11575	3.365
0:64,1:64,2:64,3:64,4:64,5:64,6:128,7:128	11575	3.368
0:64,1:64,2:64,3:64,4:64,5:64,6:128,7:128,8:128	11575	3.368
0:64,1:64,2:64,3:64,4:64,5:64,6:128,7:128,8:512,9:512	11575	3.368
full causal (no windowing)	8276	3.379
0:64,1:64,2:128,3:128	8276	3.372
0:64,1:64,2:64,3:64,4:128,5:128,6:512,7:512	8276	3.373
0:64,1:64,2:128,3:128,4:512,5:512	8276	3.373
0:64,1:64,2:128,3:128,4:256,5:256,6:512,7:512	8276	3.374
0:128,1:128,2:128,3:128,4:512,5:512	8276	3.377

Table 4: Per-head attention span sweep, with a schedule-matched causal baseline in each iteration-budget group. Every transformer block gets the same array of per-head windows. Wall clock omitted due to runtime contention with another experiment on the machine.

The addition of length-64 context window heads has significantly improved convergence under the same training regime/iteration budget, and as such, was the only architectural change to bring measurable improvement in convergence.

The actual final training recipe uses a depth-varying window cascade, motivated by the fact the model likely does not initially summarize across long distances. This also makes the forward pass slightly faster without reducing expressivity. I have not properly ablated these per-block varying context windows, so it’s not a good research practice, but introducing it in the final recipe helped save a tiny bit of time.

Blocks	$w = 64$	$w = 128$	$w = 256$	$w = 512$	$w = 1024$ (global)
0 – 1 (shallow)	6	2	2	2	0
2 – 3	4	2	2	2	2
4 – 11 (rest of depth)	4	2	0	0	6

Table 5: Per-head attention span cascade used in the final recipe. 4 heads stay local ($w = 64$) at every depth, the remaining heads widen with block index.

4 TRAINING OPTIMIZATIONS

Besides modifying the model architecture itself, I ended up heavily modifying the training procedure. First, after trying to tackle the context-length optimization through training schedules, I discovered a potential efficiency improvement in gradient noise by varying the global batch size, and then applied an EMA over the model weights to test if it further improves the noisier but faster training. Lastly, I introduced an additional L2 loss term which is somewhat similar to next-token prediction but from a metric point of view.

4.1 BATCH-SIZE SCHEDULING

Initially, I conjectured that the model does not benefit from the full 1024 context length at random initialization of the weights. Intuitively, when latents are first settling, the prediction of tokens over short context spans should be sufficient to give the model a “rough idea”. Only later in training, when the loss drops to a certain level, the model is shown longer and longer sequences. This kind of scheduling affects training in 2 additional ways, the total token count per batch (GD step) and the runtime, because shorter sequences lead to significantly faster forward passes ($O(T^2)$). I tested a couple of hand-picked schedules to assess the runtime gain versus loss in model quality.

Even though the problem’s README says that hyperparameter magic *should not* be the approach taken to speed things up, I stand my case that I arrived at this optimization through principled reasoning about gradient estimator noise, and as such, it is a valid optimization of the training algorithm which is among the things permitted by the README.

Sequence-length schedule (training fraction $\{:\}T$)	Iters	Grad accum	Val loss (final)	Wall
$T = 1024$ (baseline)	4768	32	3.379890	149.6 min
0:64,0.05:128,0.15:512,0.25:1024	4768	32	3.722735	87.4 min
0:256,0.5:1024,0.95:2048	4768	16	3.391295	150.3 min
0:64,0.2:128,0.4:256,0.7:1024,0.9:2048	4768	16	3.733675	108.4 min
0:64,0.05:128,0.2:256,0.5:1024,0.9:2048	4768	16	4.035129	39.8 min
0:64,0.05:128,0.2:512,0.5:1024,0.9:2048	4768	16	3.627057	97.4 min

Table 6: Sequence-length schedule sweep. Only 0:256,0.5:1024,0.95:2048 (the gentlest ramp, most time at $T = 1024$) comes close to the target. Faster schedules underperform.

Across the board, this approach failed. Curiously, the model initially converged to a set loss threshold significantly faster than the baseline. I tried to reason why, and ended up quickly testing whether this is only a function of tokens per step, or also the context length. As supported by the following comparison of 2 training runs, modulated tokens per step at full context length outperform modulated context length. Tokens-per-step schedule (ramp) modulated via gradient accumulation due to reasons explained in the last paragraph.

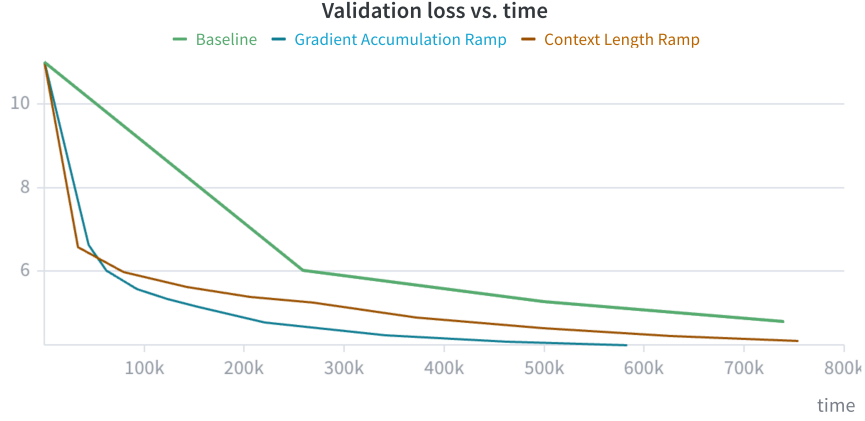


Figure 5: Validation loss over absolute training wall-clock time. Context length ramp initially wins, as the forward pass is significantly cheaper. Both beat baseline.

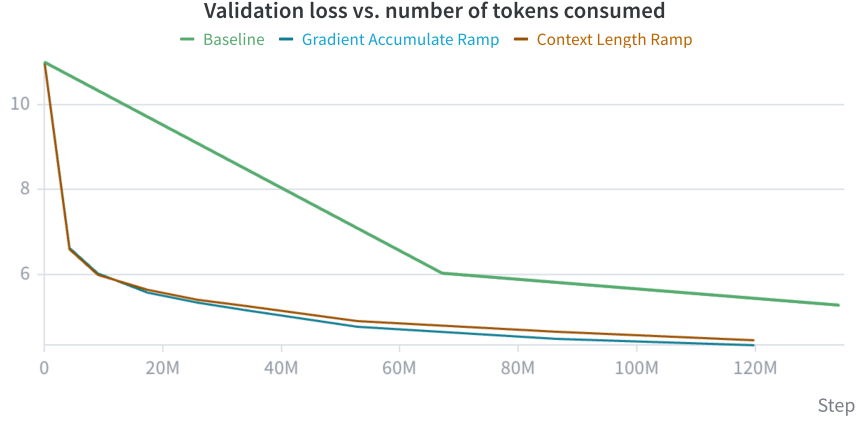


Figure 6: Validation loss over total number of tokens consumed by the training process. Tokens-per-step ramp quickly edges out context length modulation. Both beat baseline.

Step	Tokens/step	Context Length Ramp (a)	Gradient Accumulate Ramp (b)
0	32,768	10.993	11.005
128	32,768	6.589	6.633
640	65,536	5.250	5.161
1152	262,144	4.338	4.237

Table 7: **Validation loss** at matched tokens/step: (a) reaches each tokens/step value by widening context T ($64 \rightarrow 128 \rightarrow 512$) at fixed $ga = 32$, while (b) reaches the same values by raising ga ($2 \rightarrow 4 \rightarrow 16$) at fixed $T = 1024$. Both runs were killed before completion (last steps 2560/4768 and 1152/4768 respectively), so this table covers only the overlapping range both reached.

Takeaway from the experiment was that both tokens-per-step scheduling and context-length scheduling reach low loss significantly faster than the baseline, however, context-length scheduling loses its raw wall clock advantage quickly. It additionally shows that to obtain

an (equally) good model early on, **significantly noisier gradients** can be used in the optimization procedure. In plain terms, this means the baseline run is wasting computation by having unnecessarily large batch size at the beginning, using low-noise gradient estimators even though ones with higher noise levels would be sufficient. **This was the main area for wall-clock improvement.** In the end, token-per-step scheduling at fixed $T = 1024$ was the branch I followed further.

Batch B	Grad accum	ms/step	Peak mem
16	16	940.4	10.6 GB
32	8	948.5	18.0 GB
64	4	911.5	31.8 GB
128	2	935.0	62.3 GB (invalid for 5090)

Table 8: Per-step throughput at fixed 262,144 tokens/step, 40-step microbenchmark.

The last step was ensuring tokens-per-step are modulated in the most efficient way possible wrt. the underlying hardware. Because gradient accumulation is always performed sequentially, more accumulation steps linearly scale up the wall clock time. There is also the option of increasing the physical batch size from the baselines’ $B = 16$, which should remain somewhat constant due to a GPU’s parallelism, and is supported by the measured data. To remain backwards compatible with an RTX 5090, I kept memory usage under 32 GB.

Run	B	Grad Acc.	Tokens/step	Total tokens	Val loss	Wall clock	First cross
baseline	16	32	524,288	2.50 B	3.379	149.6 min	148.6 min
low B , high ga	16	16	262,144	2.17 B	3.379	131.3 min	130.0 min
high B , low ga	64	4	262,144	2.17 B	3.379	127.1 min	125.8 min

Table 9: Runtime comparison at matched tokens/step, both runs process the identical 262,144 tokens/step and identical total tokens (8276 iters), differing only in the B /ga split. $B=64$ is 3.6% faster wall clock for statistically identical loss. First cross is the first **eval step at or below the 3.3821 target.**

Clearly, a lot could be gained by optimizing the batch size to better exploit the hardware, shrink it to increase the gradient noise within acceptable levels, and make training more data (and in turn time) efficient. Finally, I combined scheduling with the hardware-optimal batch size, leading to a significant improvement in wall clock time.

Batch B	GA schedule	Tokens/step	Iters	Val loss	Wall clock	First cross
16 (baseline)	const. 32	524,288	4768	3.379	149.6 min	148.6 min
64	const. 4	262,144	8276	3.379	127.1 min	125.8 min
64	0:2,0.57:4	first 131,072 then 262,144	11575	3.377	127.5 min	124.7 min

Table 10: First run combining the gradient accumulation ramp with $B = 64$: ga = 2 (131,072 tok/step) for the first 57% of training, then ga = 4 (262,144 tokens/step) for the rest. “First cross” defined identically to the previous table. Schedule crosses sooner than constant.

This reasoning and optimization process brought a significant speedup of 16%. As such, it was included in the final training recipe and all the following experiments.

4.2 PASSIVE EXPONENTIAL MOVING AVERAGE ON MODEL PARAMETERS

Since noise in the gradients has shown to be an important factor in training efficiency, a natural continuation of the previous experiment was analysis of noise in the model weights, and if there is any potential to get an overall better model by averaging the parameters over the last few gradient descent steps. To evaluate this efficiently, I implemented the EMA to run in parallel with training, and allow testing of an arbitrary number of different EMA coefficients (e.g. 0.998, 0.95, 0.96) all at once. Each averaged model is evaluated during the validation portion of training, which is **excluded** from the overall training wall-clock time, so evaluating many EMAs costs no additional wall clock.

```

 $\Theta = \{\theta_d : d \in D\}$ , shadow copies indexed by decay  $d$ , init  $\theta_d \leftarrow \theta$ 

for  $t = 0, \dots, T$ :
     $g_t \leftarrow \text{accum}(s, \frac{t}{T})$                                 piecewise schedule  $s$ 
    if  $t \bmod E = 0$ :
         $\ell \leftarrow L(\theta)$ 
        for  $\theta_d \in \Theta$ :
             $\theta, \theta_d \leftarrow \theta_d, \theta$                         swap in
             $\ell_d \leftarrow L(\theta)$ 
             $\theta, \theta_d \leftarrow \theta_d, \theta$                         swap out
         $\mathcal{L} \leftarrow \frac{1}{g_t} \sum_{i=1}^{g_t} \text{loss}(x_i, y_i, \theta)$         compute loss
         $\theta \leftarrow \theta - \eta_t \cdot \nabla_{\theta} \mathcal{L}$                 opt. step
        for  $\theta_d \in \Theta$ :
             $\beta_t \leftarrow \min(d, \frac{t}{t+1})$                         bias-correction ramp
             $\theta_d \leftarrow (1 - \beta_t) \cdot \theta + \beta_t \cdot \theta_d$     convex combination

```

Table 11: Training-loop core as a convex-combination EMA update over persistent/averaged parameter sets θ_d , with swap-based evaluation (to only change parameters of a static `torch.compile` kernel without re-compilation) and a piecewise grad-accumulate schedule g_t as a general form of method presented in Table 10.

First, I tested some long-horizon values, this experiment was interleaved with the scheduling experiment, so the initial test was with a constant tokens-per-step schedule.

EMA decay	Horizon ($\frac{1}{1-d}$)	Val loss	vs. raw	First cross
raw (no EMA)	–	3.380037	–	130.0 min
0.998	500 steps	3.380715	+0.0007	130.0 min
0.999	1000 steps	3.387231	+0.0072	never
0.9995	2000 steps	3.419085	+0.039	never
0.9998	5000 steps	3.573685	+0.194	never

Table 12: Long-horizon EMA sweep, $B = 16$, $ga = 16$ (constant, no schedule), 8276 iters, 131.3 min wall clock time. First cross is the first eval step at or below 3.3821, raw and EMA 0.998 both cross at the same step (8192/8276) since they track almost identically at this short horizon. Loss numerical precision increased to show fine differences.

Clearly, the trend indicated that the shorter the schedule, the lower the loss (model improves rapidly during the warmdown phase and long horizon average produces worse parameters). I

took the then-best performing EMA of 0.998 and added it to the fastest-crossing experiment from Table 10.

EMA decay	Horizon ($\frac{1}{1-d}$)	Val loss	vs. raw	First cross
raw (no EMA)	–	3.377058	–	124.7 min
0.998	500 steps	3.377723	+0.0007	122.7 min

Table 13: $B = 64$, grad-accumulate schedule 0:2,0.57:4, 11575 iterations, 127.5 min. EMA 0.998 finishes essentially tied with raw (+0.0007, same gap as in Table 12) but crosses the target 2.0 minutes earlier.

For my later tests, I decided to rather sweep over aggressively short horizons. The results have shown it’s more likely averaging horizons in orders of tens to low hundreds of iterations will outperform the raw model, so I continued with those values instead.

EMA decay	Horizon ($\frac{1}{1-d}$)	Val loss	vs. raw	First cross
raw (no EMA)	–	3.378729	–	113.3 min
0.95	20 steps	3.378752	+0.00002	113.3 min
0.96	25 steps	3.378763	+0.00003	113.3 min
0.97	33 steps	3.378784	+0.00006	113.3 min
0.98	50 steps	3.378831	+0.00010	112.4 min
0.99	100 steps	3.378919	+0.00019	112.4 min

Table 14: $B = 64$, grad-accumulate schedule 0:2,0.57:4, attention windows, auxiliary L2 loss, 10566 iters, 115.3 min. Final loss is monotonically worse with longer horizon, but 0.98 and 0.99 cross the target one evaluation step earlier (10368 vs 10432) than raw and the shorter horizons.

Both experiments have shown that short-horizon EMA on the model parameters can converge to the threshold loss faster, and because it does not increase the apparent wall-clock time, I decided to keep the sweeps active during all following training runs, and then simply picked **whichever EMA crossed the target loss threshold first**.

4.3 AUXILIARY L2 EMBEDDING LOSS

Having exhausted the architectural direction, and the optimization procedure, my attention turned to the loss function itself. I recalled some knowledge from a couple of years ago, when I was interested in *implicit layers* (Duvenaud et al., 2020). I was curious how it would look like if we were to unroll a transformer “to infinity”, i.e., have a “fixed-point attention block”, that would output the true next-token embedding in one step. Denote the input token embedding $\mathbf{x}$, and its ground-truth next-token counterpart $\mathbf{h}^*$. Then the “implicit transformer” works, very roughly, as $\mathbf{h}^* = \text{Block}(\mathbf{h}^*, \mathbf{x})$. where for brevity this “implicit block” would be the last one in the otherwise finite model.

Implicit layers can involve multiple iterations, or implicit formulations through Jacobian inversions, neither of which are helpful in the context of minimizing training time.

Instead, a very simple approximation can be made, which **adds a metric regularizer** to the standard cross-entropy objective, and turns the implicit transformer into a “regression problem”. Consider the model’s output $\hat{\mathbf{h}}$ as an approximation of $\mathbf{h}^*$. Because the model shares the same embedding matrix $\mathbf{E}$ on both input and output, we know that $\mathbf{h}^* = \mathbf{E}_{y_t}$, where y is the index of the ground-truth next token in the vocabulary of the model. Cross-entropy loss maximizes the probability on the correct token, whose raw logits are simply dot products $\hat{\mathbf{h}} \cdot \mathbf{E}_{i,:}$ for all i in the vocabulary. Maximizing probability through softmax directly maximizes the corresponding logit (and minimizes others), i.e. maximizes the dot product. We can also maximize the dot product between the model output and the correct embedding by **minimizing their L2 distance**. This is exactly the additional loss term I experimented with.

Formally, let $\mathbf{E} \in \mathbb{R}^{V \times d}$ be the (tied) embedding matrix, $\hat{\mathbf{h}}_t \in \mathbb{R}^d$ the model’s output hidden state at position t , and y_t the index of the ground-truth next token. The implicit-transformer fixed point from above is approximated by a single forward pass, $\hat{\mathbf{h}}_t \approx \mathbf{h}_t^*$, and because the head is tied to $\mathbf{E}$, the target itself is just an embedding row:

$$\mathbf{h}_t^* = \mathbf{E}_{y_t,:}. \quad (2)$$

The standard cross-entropy objective operates on logits $\ell_{t,i} = \hat{\mathbf{h}}_t \cdot \mathbf{E}_{i,:}$ for every vocabulary entry i , and softmax maximizes ℓ_{t,y_t} relative to the rest, i.e. it maximizes the dot product $\hat{\mathbf{h}}_t \cdot \mathbf{E}_{y_t,:}$. Since

$$\|\hat{\mathbf{h}}_t - \mathbf{E}_{y_t,:}\|_2^2 = \|\hat{\mathbf{h}}_t\|_2^2 + \|\mathbf{E}_{y_t,:}\|_2^2 - 2 \cdot \hat{\mathbf{h}}_t \cdot \mathbf{E}_{y_t,:}, \quad (3)$$

minimizing the left-hand side also maximizes that same dot product, so the same objective can be reframed as a metric regression problem, pulling $\hat{\mathbf{h}}_t$ toward the (stop-gradented) target embedding directly, in L_2 . Note that the two norm terms are not both constants. The target norm is, since the row is detached, but $\|\hat{\mathbf{h}}_t\|_2^2$ is not, so the term is dot-product maximization plus an explicit penalty on the output norm. Cross-entropy only depends on logit differences and is therefore free to grow $\|\hat{\mathbf{h}}_t\|$ without bound.

Averaging over the N non-masked positions in a batch and over the d embedding dimensions:

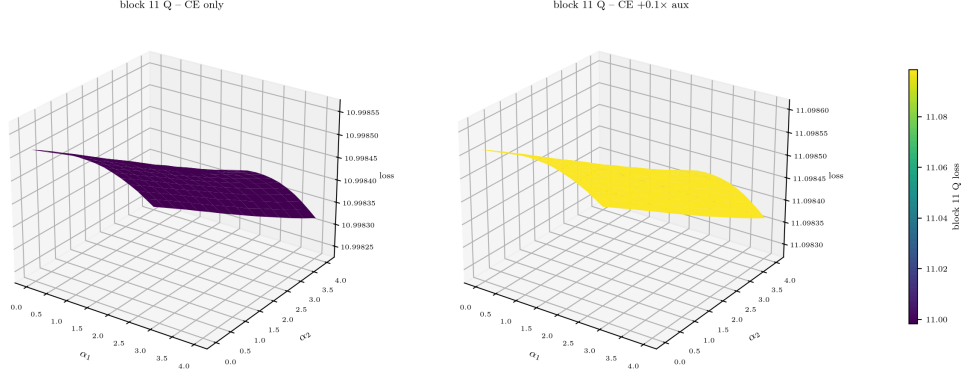
$$\mathcal{L}_{\text{aux}} = \frac{1}{Nd} \sum_{t=1}^N \|\hat{\mathbf{h}}_t - \text{detach}(\mathbf{E}_{y_t,:})\|_2^2, \quad (4)$$

where t enumerates non-masked positions over the flattened batch. The training objective becomes

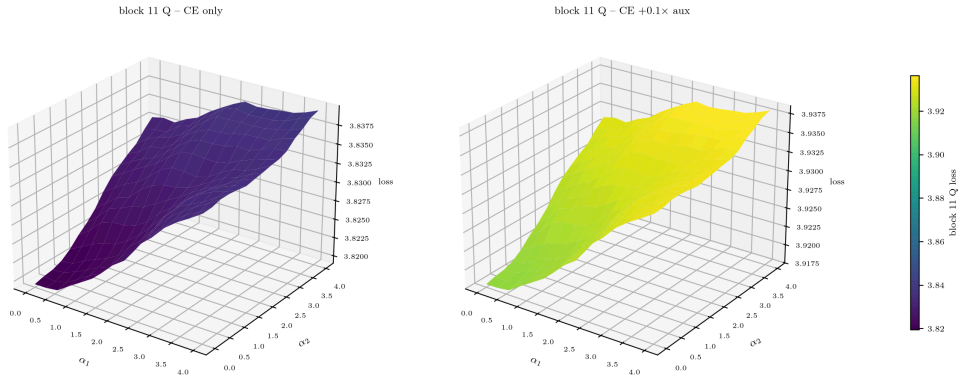
$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda \cdot \mathcal{L}_{\text{aux}}, \quad \lambda \in [0, 0.5] \text{ (in my experiments)} \quad (5)$$

I wanted to see the effect of this additional term on the overall loss landscape graphically, and as such, I decided to plot the raw cross-entropy loss versus the regularized one. To construct the plots, I took a fresh-initialized checkpoint, a checkpoint halfway through training, and the final model. This was critical, as the loss depends on the values of $\mathbf{E}$ which

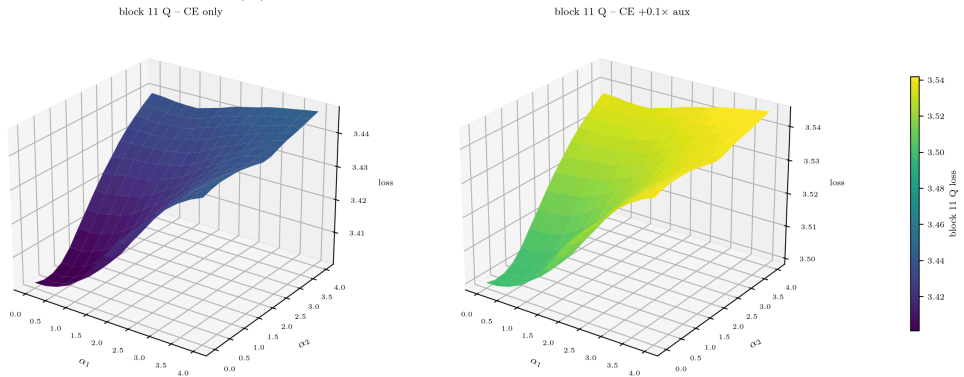
change throughout training. I then sampled random normal matrices A, B, C, D , where A, B would be added to the Q, K, V projection matrices as a conic perturbation, e.g. $Q' = Q + \alpha_1 A + \alpha_2 B$, $\alpha_1, \alpha_2 > 0$. Note the perturbation was normalized as per Li et al. (2018). The same approach was applied to the MLP projection matrix, just with C, D as the direction matrices. With the modified weight matrix (one at a time), I ran the forward pass of the model and computed loss against the provided validation split of FineWeb. This gave a 3D landscape loss plot, with α_1, α_2 as x, y coordinates and loss making up the z .



(a) freshly-initialized model



(b) model midway through training



(c) final trained model

Figure 7: Loss landscape evolution across training, **Query matrix in transformer block #11**. Left is pure cross-entropy, right is cross-entropy and the auxiliary term added up. Color-value scaling shared between the plots. Auxiliary term ends up manifesting almost solely as a constant offset of the loss.

First, looking at the loss landscapes when perturbing the attention projection matrices (specifically $\mathbf{Q}$ in Figure 7), the auxiliary L2 term introduces a constant offset of the loss landscape, but does not affect the geometry (landscape smoothness) at all.

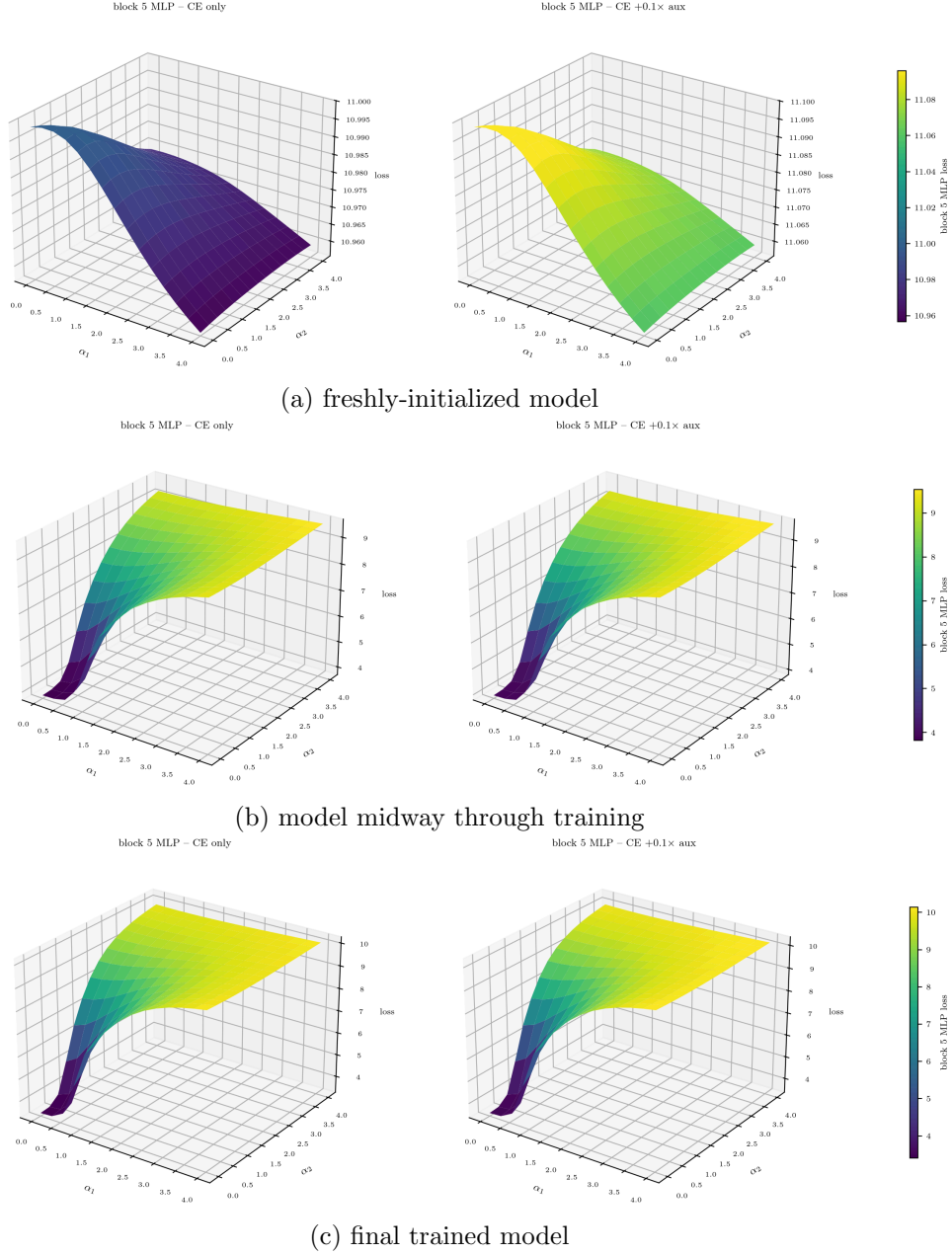


Figure 8: Loss landscape evolution across training, **MLP projection matrix in transformer block #5**. Left is pure cross-entropy, right is cross-entropy and the auxiliary term added up. Color-value scaling shared between the plots. Auxiliary term ends up minimal or only offsets the loss value. The landscape geometry remains essentially identical.

In my experiments, I ran a sweep of values of λ from 0.025 to 0.5. The additional loss term did not bring overly significant improvements, however, they were still measurable and ultimately led to couple of percent of reduction in overall training time. It is possible that the improvement is in the gradient direction as whole, and cannot be really (visually) inferred from any low-dimensional, isolated perturbation.

λ	Iters	Val loss	Wall	First cross
0 (curr. baseline)	10566	3.379	117.0 min	116.0 min
0.5	10566	3.380	114.9 min	113.9 min
0.2	10566	3.379	115.0 min	113.9 min
0.1	10566	3.378	114.4 min	111.5 min
0.05	10566	3.379	115.0 min	113.9 min
0.025	10566	3.367	115.0 min	114.0 min

Table 15: Aux-loss coefficient (λ) sweep across different values, compared to then-baseline (previous optimizations already applied). “First cross” is first evaluation step at or below 3.3821 on any logged metric (raw or EMA). Evaluation steps spaced apart by about 1 minute and 64 training steps. In bold: overall fastest crossing and overall best final loss. Addition of the auxiliary loss term always accelerates first crossing of validation CE loss.

The observed behavior suggests the additional loss term “accelerates” the model towards the optimum of cross-entropy in every λ tested, however, once close to it, starts pulling the parameters away/overshoots, and ultimately degrades the model to a worse final loss than pure cross-entropy. This is in line with the final loss slightly growing with increasing λ . The crossing time 111.5 minutes is the fastest runtime achieved overall, and therefore became the final training recipe.

5 FINAL TRAINING RECIPE

The final training recipe combines the limited attention windowing described in Section 3.4, specifically, it uses the per-layer varying configuration described in Table 5. PyTorch’s `flex_attention` measurably benefits from shorter attention windows in terms of raw computation time, pairing that with the short-range assumption appeared to be a good direction for saving some time.

Stat	Baseline	Fastest cross
Batch B	16	64
Grad accum	const. 32	0:2, 0.57:4
Iterations	4768	10566
Warmup / warmdown	256 / 1024	571 / 4543
Attention windows	dense causal	depth-varying cascade (Table 5)
EMA decay(s)	none	0.98 , 0.97, 0.96
Aux λ	0	0.1
Final val loss	3.379	3.378
Total training time	149.6 min	114.4 min
First cross (target 3.3821)	148.6 min	111.5 min
Total tokens processed	2.500 B	1.980 B
Tokens to first cross	2.483 B	1.928 B
Training-time speedup vs. baseline	–	23.5 %
First-cross speedup vs. baseline	–	25.0 %

Table 16: Baseline vs. the fastest-crossing recipe, the actual speedrun winner by the project’s objective (minimize training time to cross 3.3821). EMA decay responsible for first crossing highlighted in bold. All unlisted hyper-parameters remain identical.

The measurable difference between the same context in every layer and the variable ones was just **24 seconds**, as both crossed the threshold in the same step, one was just slightly faster to compute.

Next, it used the batch (grad-accumulate) schedule (B=64, 0:2,0.57:4) from Section 4.1. Two independent findings stacked here. First, at a fixed 262,144 tokens/update, splitting it as B=64/ga=4 beats B=16/ga=16 on pure GPU kernel throughput with 3.6% less wall clock and statistically identical loss. Second, ramping ga from 2→4 over the first 57% of training beats holding it constant at 4. This simpler two-stage version, tried after the constant grad-accumulate tuning had already found 4 as the target, is what worked the best.

It had a slightly-shortened iteration count (10566, warmup 571, warmdown 4543), derived by proportionally shortening original 11,575-iteration schedule by about 8.7%. I arrived at this number by re-running the experiment couple of times and slowly shortening the iteration count, approx. doubling the decrease from 2% and up. Run shortened to 10% did not cross the threshold, so 8% (+ rounding errors) remained as the best. The proportion of warmup/constant/warmdown split over all of the iterations remained the same.

To track the shorter training with a smaller batch size, three EMAs (Section 4.2) (0.98, 0.97, 0.96) were used to track the model. All short horizons (25–50 steps). EMA 0.98 is what actually delivered the fastest crossing time.

Finally, the training used the embedding auxiliary loss (Section 4.3) with $\lambda = 0.1$.

6 CONCLUSION AND POTENTIAL IMPROVEMENTS

Through both training algorithm optimization and slight architectural improvements, I was able to achieve a reasonable speedup of 25%, without having to touch any hyper-parameter besides the batch size, which I adjusted in a principled and theoretically well-motivated manner to not count as “hyper-parameter magic”.

A big open question is whether these improvements can withstand stochasticity of different random seeds, and then perhaps what is the average speedup and its standard deviation. From some very rough observations, I believe the tuned recipe (Section 5) is lower variance than the baseline, which as mentioned in Section 2, occasionally had problems crossing the target threshold at all, so my expectation is they will hold up, or increase slightly on average.

Because I was testing the modifications in a linear manner (the same one as presented in this report) there has potentially been some performance left on the table, as e.g. the embedding factorization could work better with the auxiliary loss term. However, such extensive experiments were beyond my time and financial resources (approx. \$1200 in compute spent on the presented experiments alone).

REFERENCES

- Duvenaud, D., Kolter, J. Z., and Johnson, M. *Deep Implicit Layers: Neural ODEs, Deep Equilibrium Models, and Beyond*, 2020.
- Geva, M., Schuster, R., Berant, J., and Levy, O. Transformer Feed-Forward Layers Are Key-Value Memories. *Arxiv Preprint Arxiv:2012.14913*, 2021. <https://arxiv.org/abs/2012.14913>
- Li, H., Xu, Z., Taylor, G., Studer, C., and Goldstein, T. Visualizing the Loss Landscape of Neural Nets. *Advances in Neural Information Processing Systems*, 31, 2018.
- Shazeer, N. GLU Variants Improve Transformer. *Arxiv Preprint Arxiv:2002.05202*, 2020. <https://arxiv.org/abs/2002.05202>